



Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

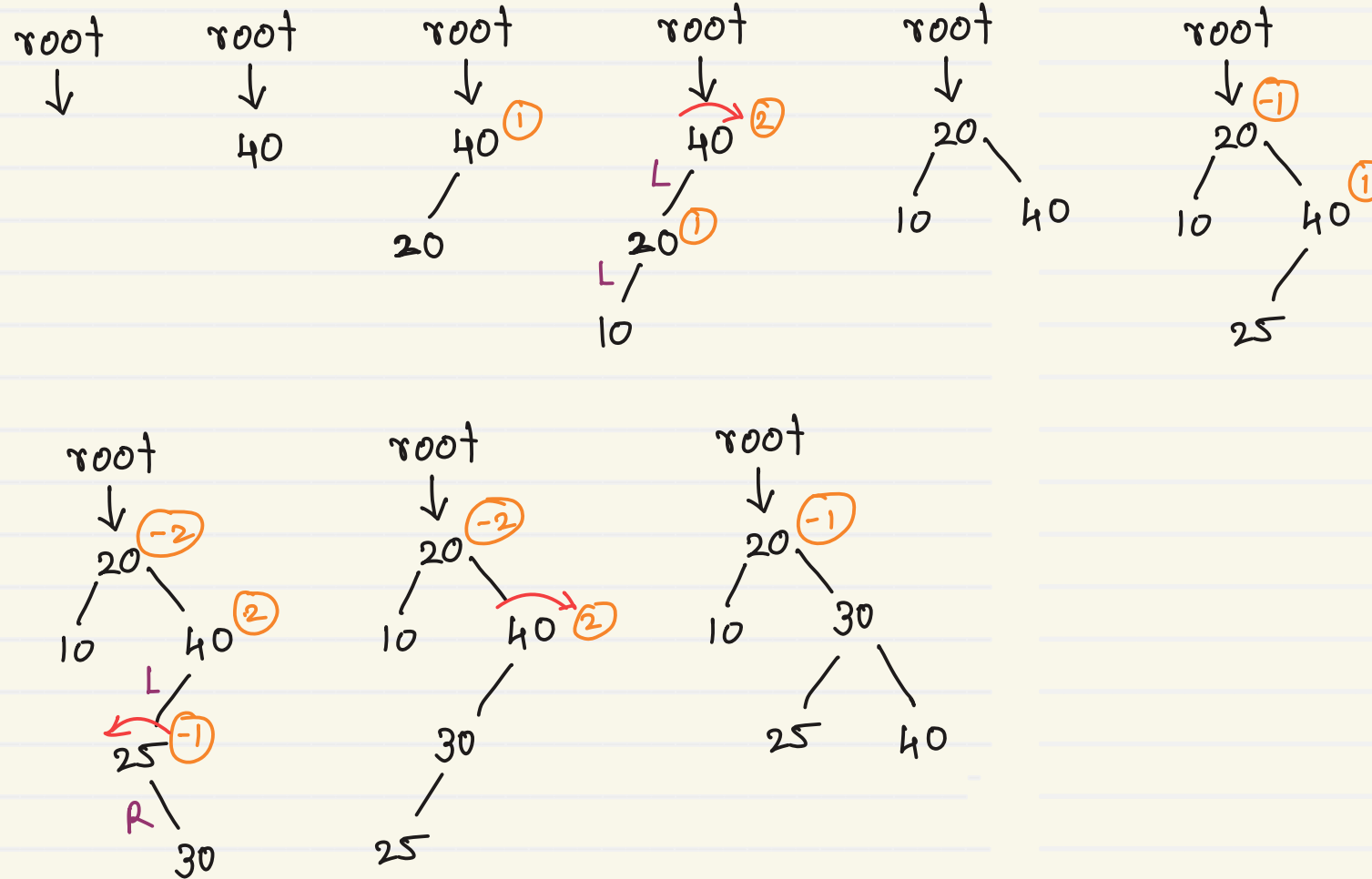
Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

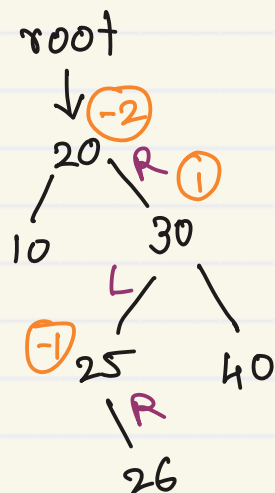
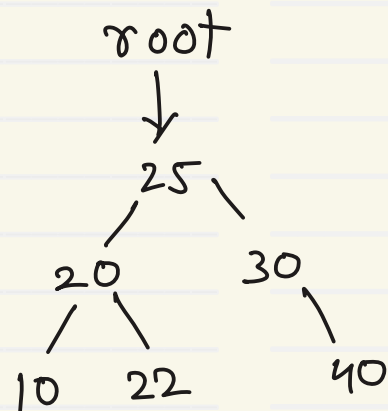
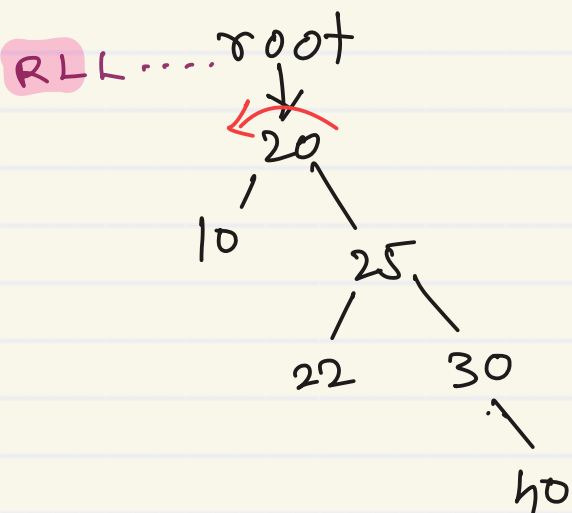
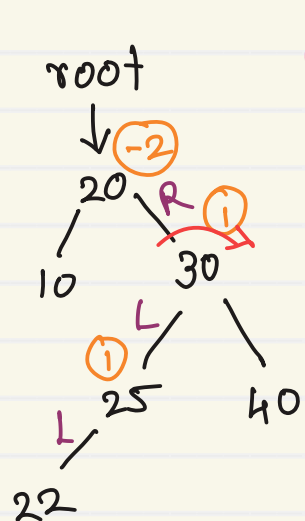
- self balancing binary search tree
- on every insertion and deletion of a node, tree is getting balanced by applying rotations on imbalance nodes
- The difference between heights of left and right sub trees can not be more than one for all nodes
- Balance factors of all the nodes are either -1 , 0 or +1
- All operations of AVL tree are performed in $O(\log n)$ time complexity

AVL Tree - Add Node

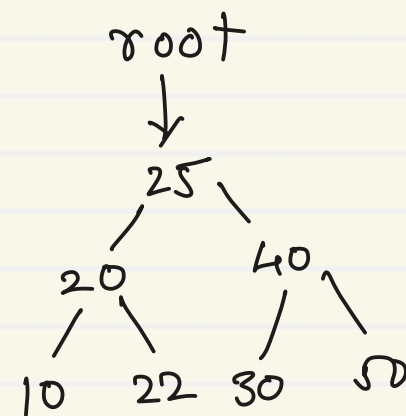
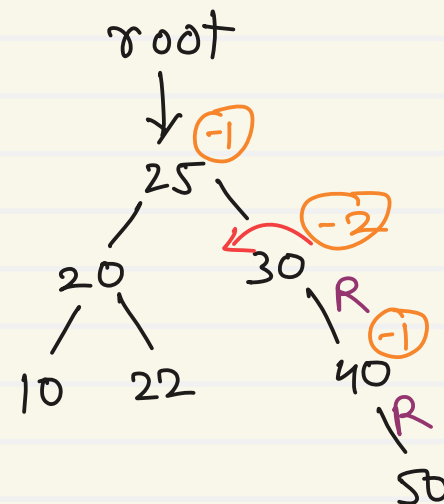
Keys : 40, 20, 10, 25, 30, 22, 50



Keys : 40, 20, 10, 25, 30, 22, 50

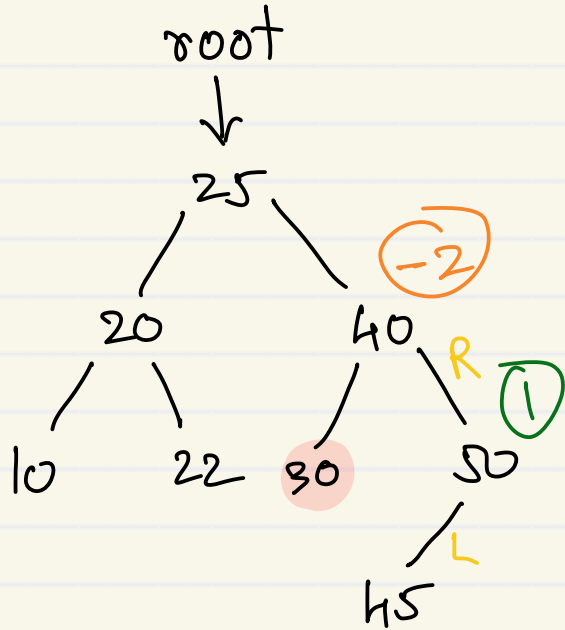


RLR...

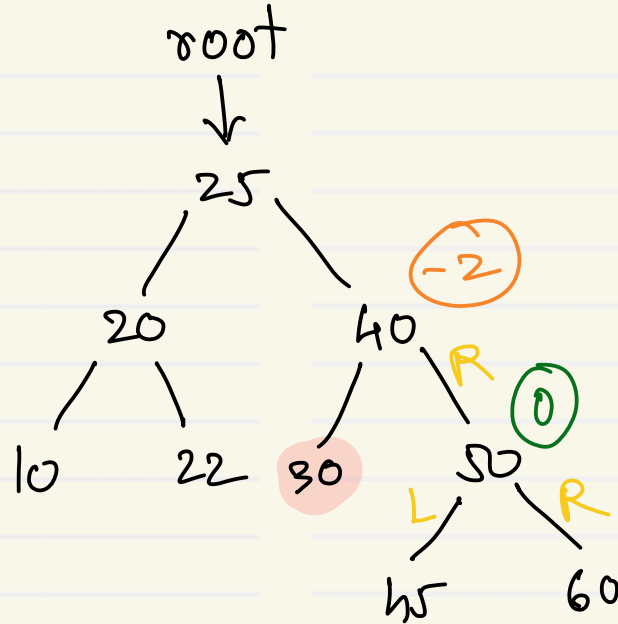


AVL Tree - Delete Node (L-Deletion)

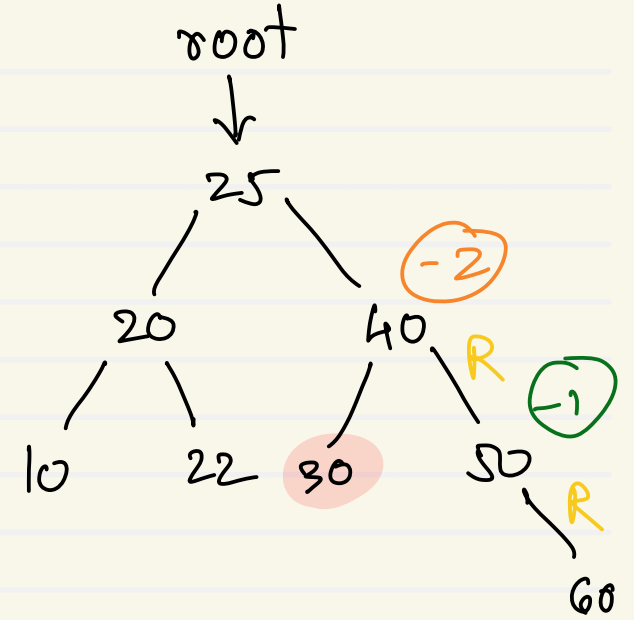
bf < -1



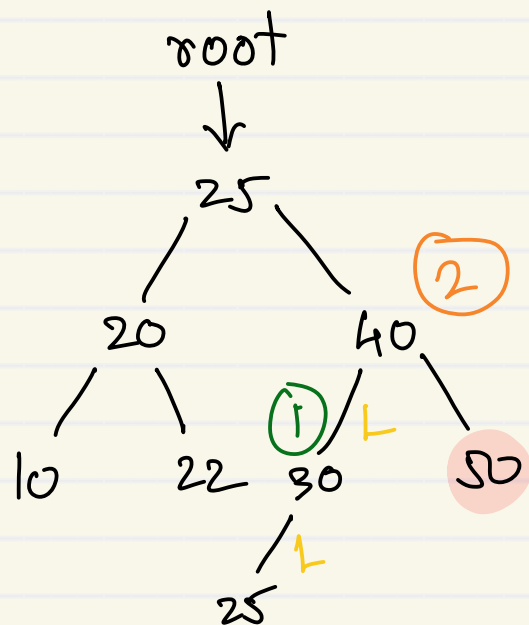
LI
↓
RL



LO
↓
RR

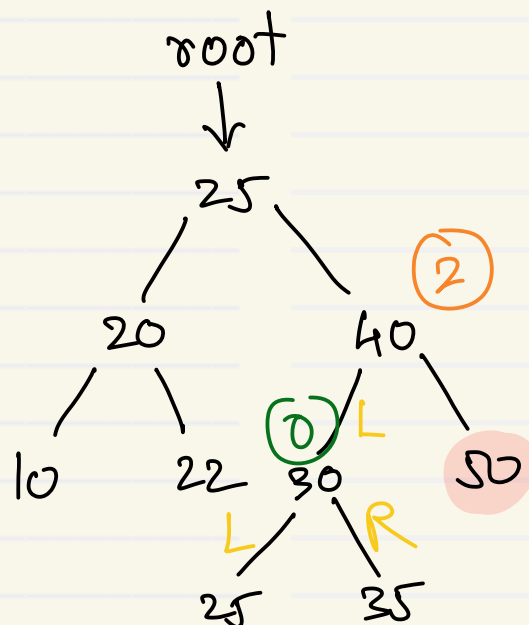


L-1
↓
RR



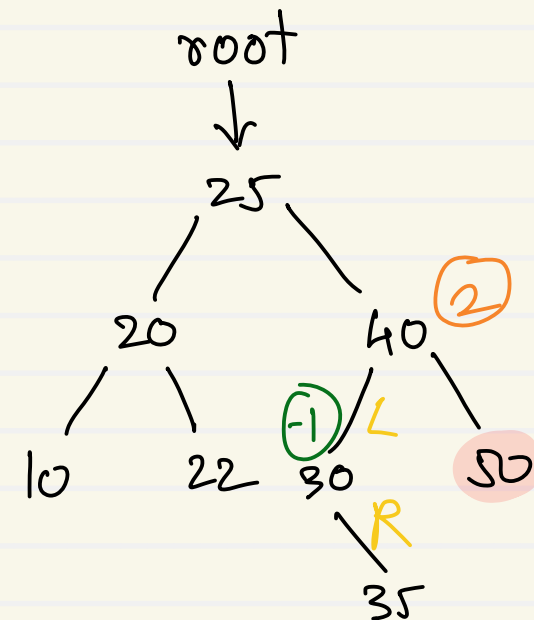
R1

↓
LL



R0

↓
LL

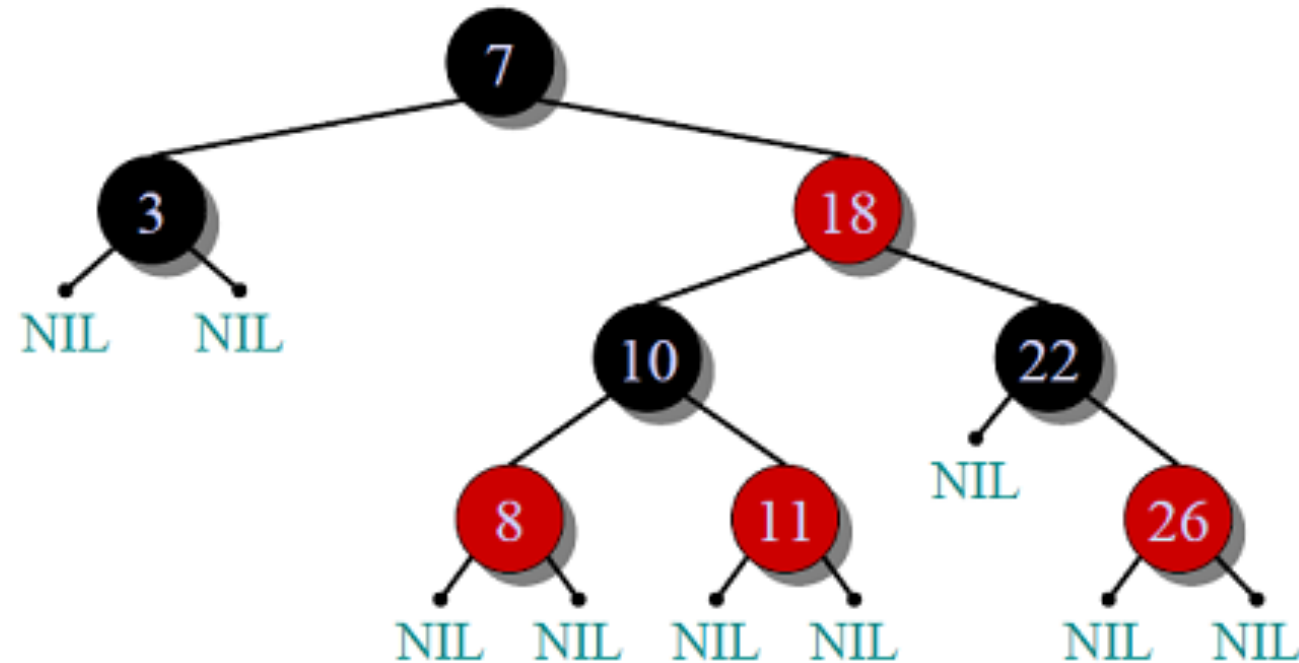


R-1

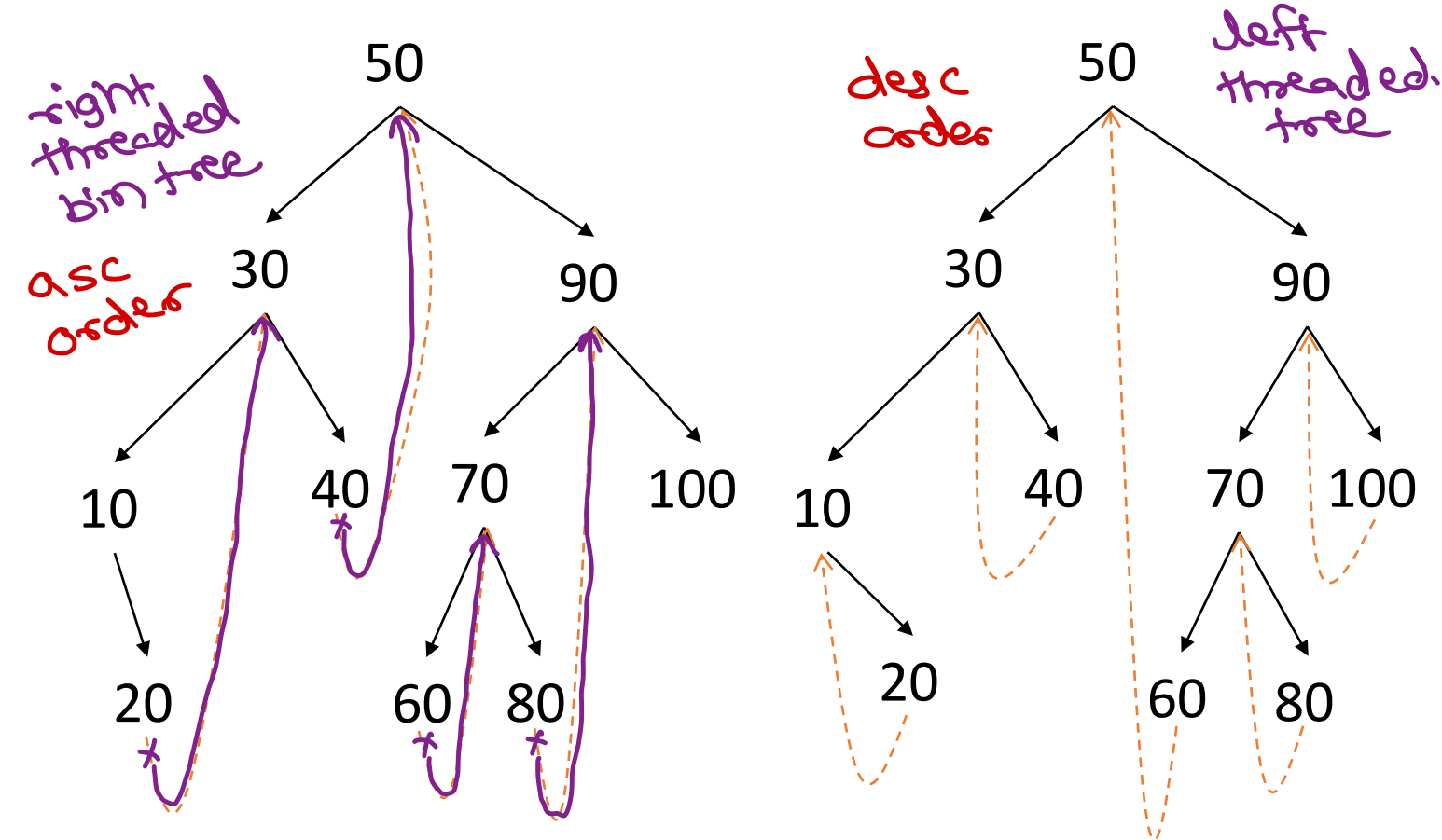
↓
LR

Red & Black tree

- Red & Black tree is a self-balancing Binary Search Tree (BST).
- Each node follows some rules:
 - Every node has a color either red or black.
 - Root of tree is always black.
 - Two adjacent cannot be red nodes (Parent color should be different than child).
 - Every path from a node (including root) to any of its descendant NULL node has the equal number of black nodes.
- Most of BST operations are done in $O(h)$ i.e. $O(\log n)$ time.
- For frequent insert/delete, RB tree is preferred over AVL tree.



Threaded BST

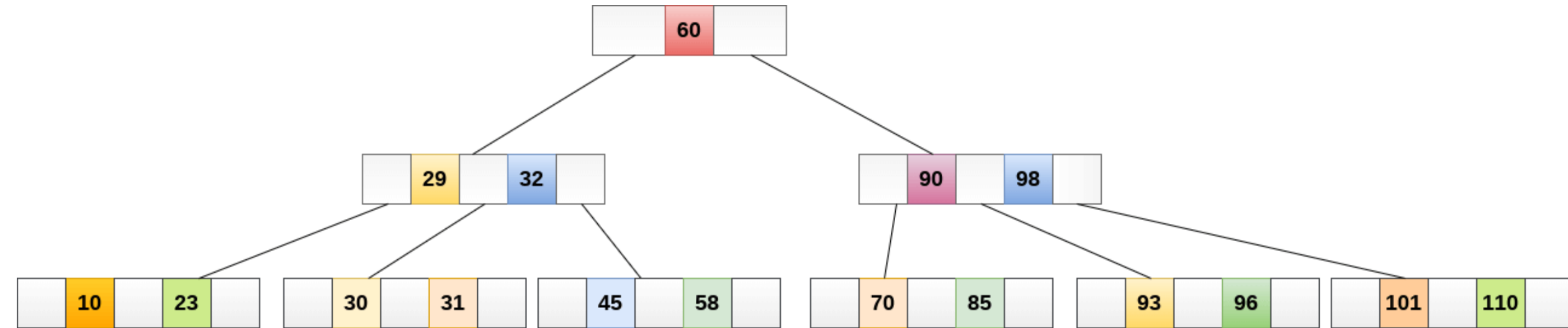


- Typical BST in-order traversal involves recursion or stack. It slows execution and also need more space.
- Threaded BST keep address of in-order successor or predecessor addresses instead of NULL to speed up in-order traversal (using a loop).
- Left threaded BST
- Right threaded BST
- In-threaded BST

right threaded BST + left threaded BST =



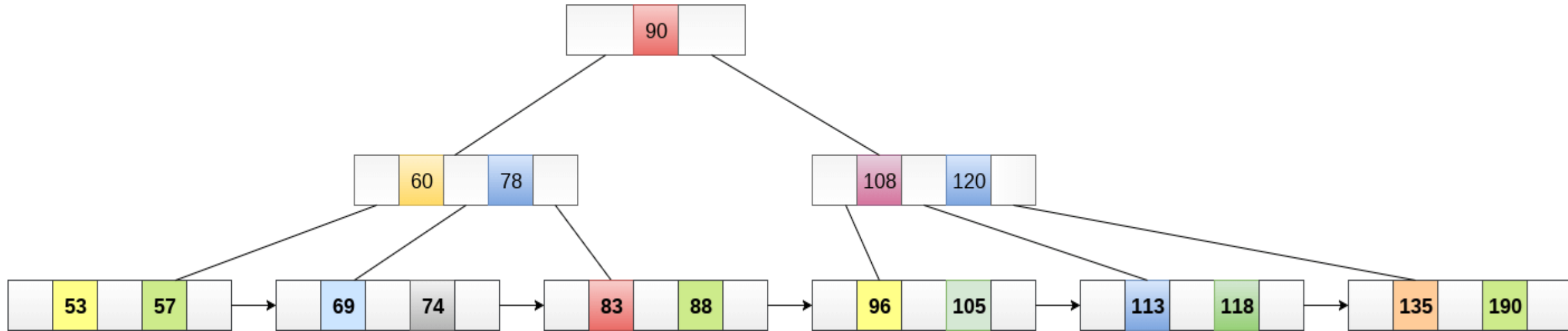
B Tree



- A B-Tree of order m can have at most $m-1$ keys and m children.
- B tree store large number of keys in a single node. This allows storing number of values keeping height minimal.
- Note that in B-Tree all leaf nodes are at same level.
- B-Tree is commonly used for indexing into file systems and databases. It ensures quick data searching and speed up disk access.



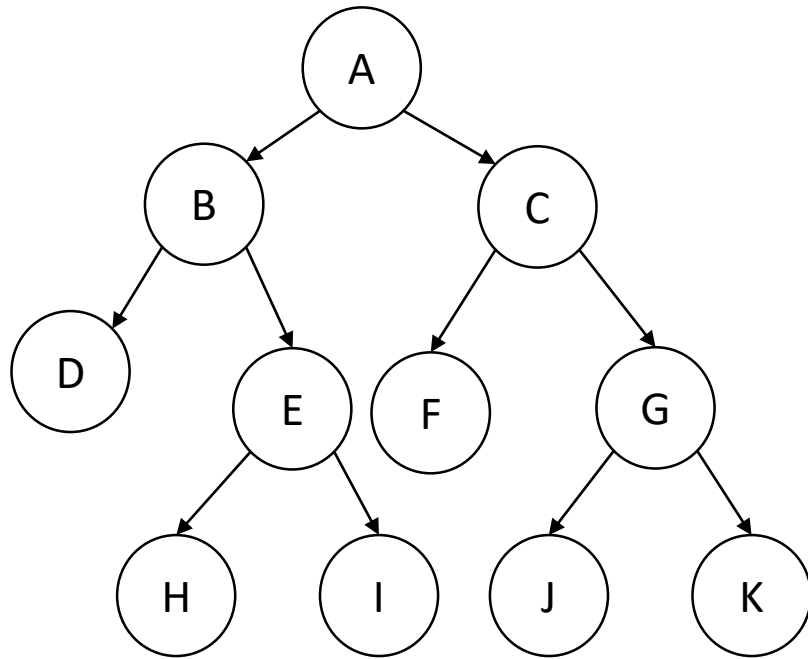
B+ Tree



- Extension of B-Tree for efficient insert, delete and search operation.
- Data is stored in leaf nodes only and all leaf nodes are linked together for sequential access.
- Search keys may be redundant.
- Faster searching, simplified deletion (as only from leaf nodes).
- B+Tree is commonly used for indexing into file systems and databases. It ensures quick data searching and speed up disk access.

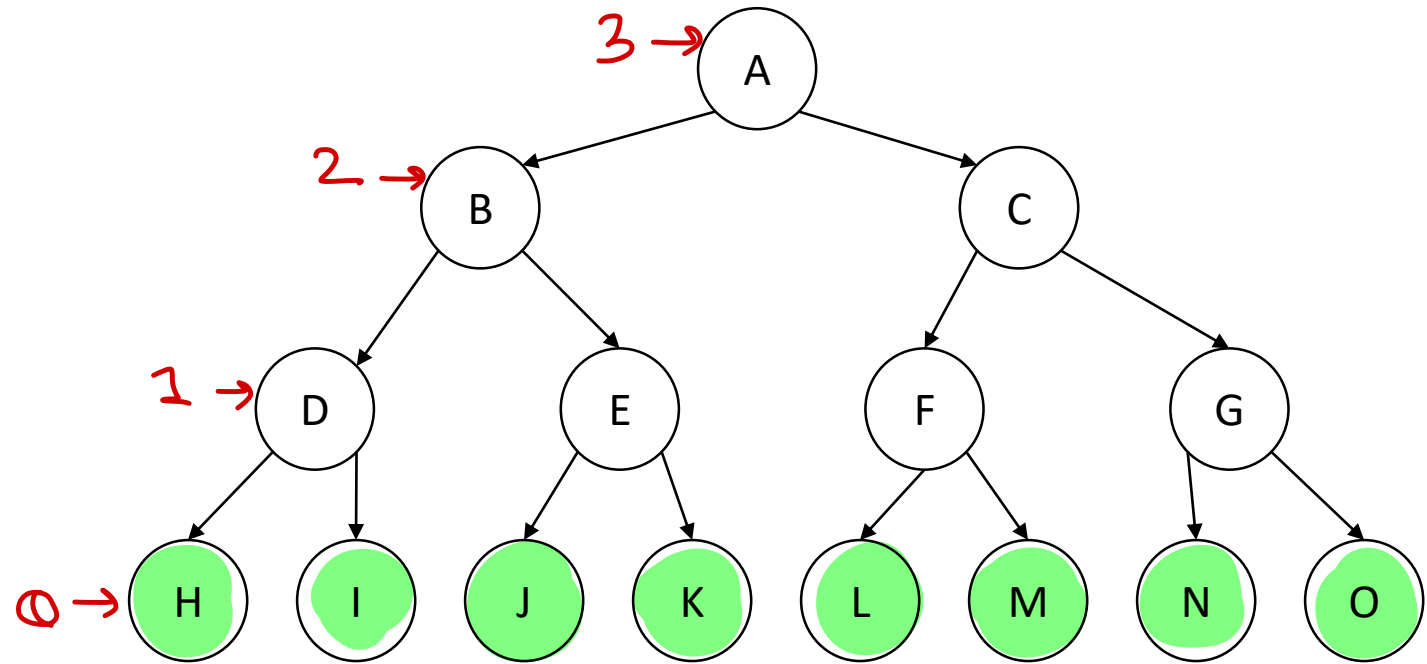


Strictly Binary Tree



- Binary tree in which each non-leaf node has exactly two child nodes.
- Strictly binary tree with n leaves always has $2n - 1$ nodes.

Complete Binary Tree



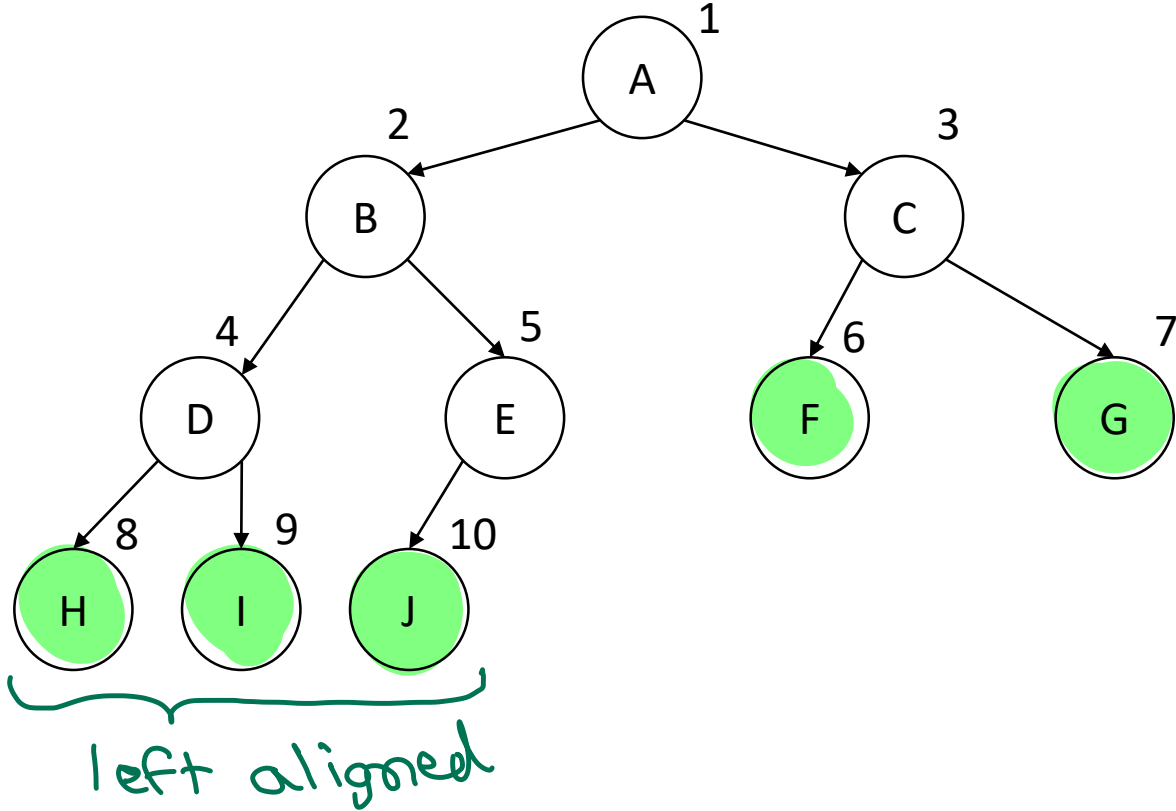
- Strictly binary tree of height h whose all leaves are at same level. Also called as full tree.

- Number of nodes = $2^{h+1} - 1$
- Number of leaf nodes = 2^h

$$\begin{aligned} 2^{3+1} - 1 &= 15 \\ 2^3 &= 8 \end{aligned}$$



Almost Complete Binary Tree and Heap

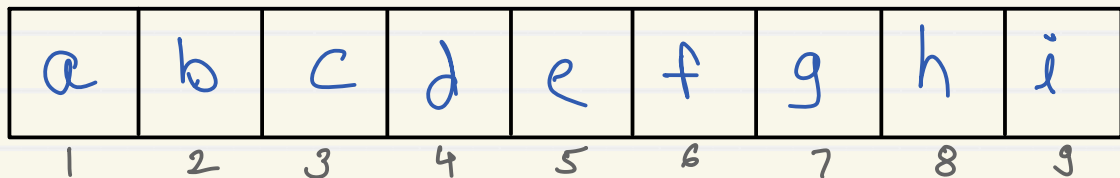
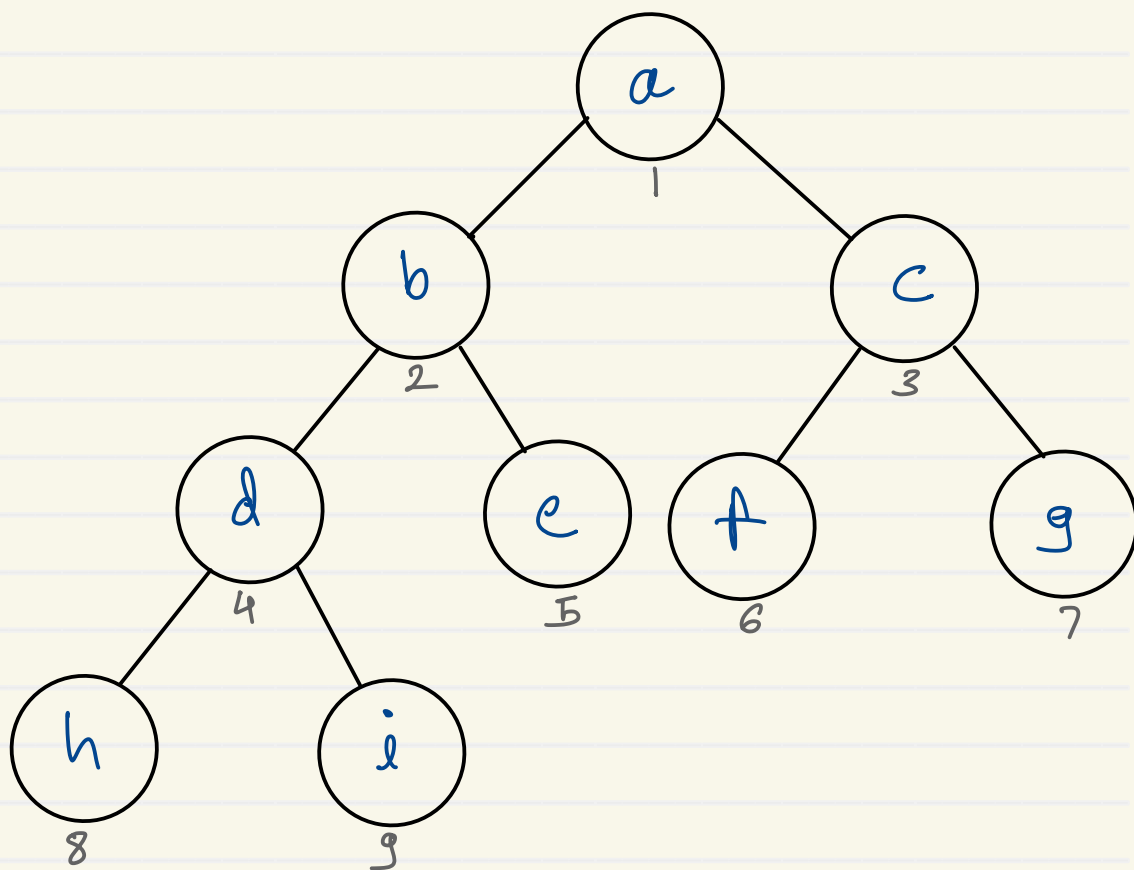


A	B	C	D	E	F	G	H	I	J
1	2	3	4	5	6	7	8	9	10

- Almost complete binary tree (of height d)
 - All leaf nodes must be at level d or $d-1$.
 - All leaf nodes at level d must aligned as left as possible.
- Heap is array implementation of almost complete binary tree.
- Parent child relation is maintained through index calculations
 - $\text{parent index} = \text{child index} / 2$
 - $\text{left child index} = \text{parent index} * 2$
 - $\text{right child index} = \text{parent index} * 2 + 1$



Almost Complete Binary Tree or Heap

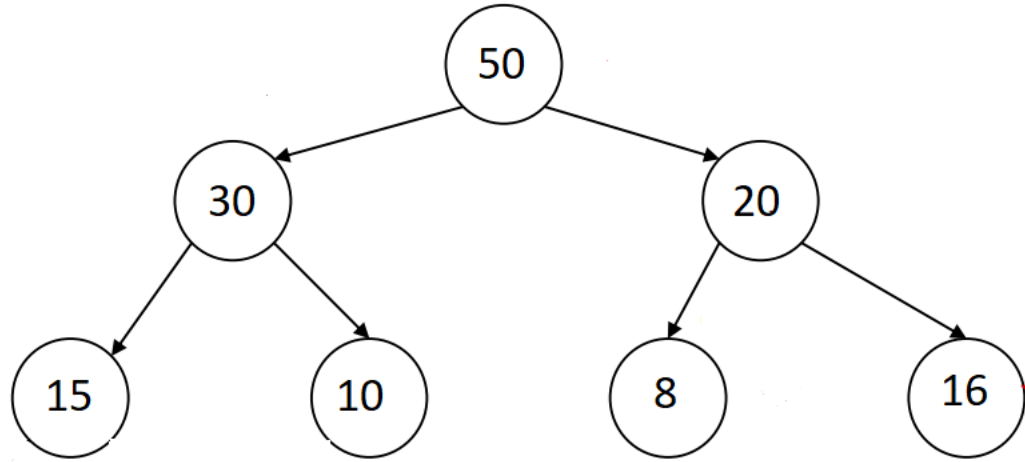


- Almost Complete Binary Tree (height = h)
- All leaf nodes must be at level h or h-1
- All leaf nodes at level h must aligned as left as possible
- Array implementation of Almost Complete Binary Tree is called as heap

Node $\rightarrow$ i^{th} index
 parent $\rightarrow$ $i/2$ index
 left child $\rightarrow$ $i*2$ index
 right child $\rightarrow$ $i*2+1$ index

Heap Types – Max and Min

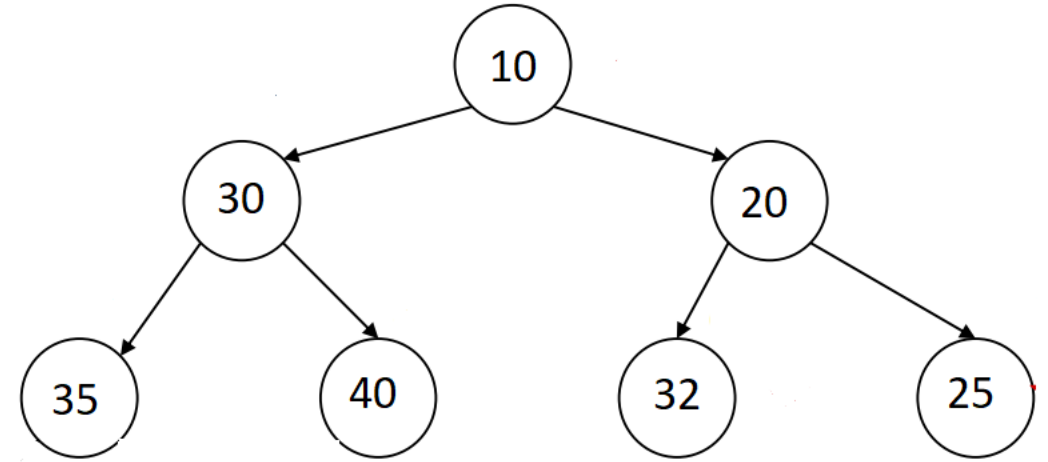
Max Heap



50	30	20	15	10	8	16
1	2	3	4	5	6	7

- Max heap is a heap data structure in which each node is greater than both of its child nodes.

Min Heap

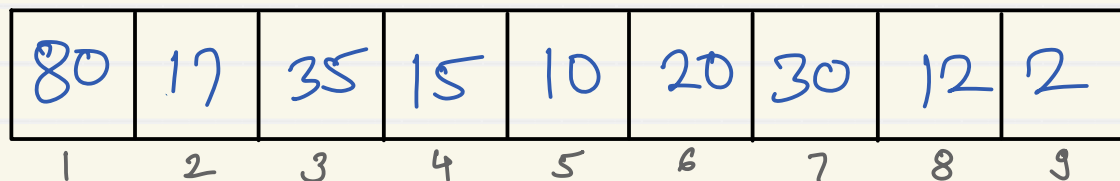
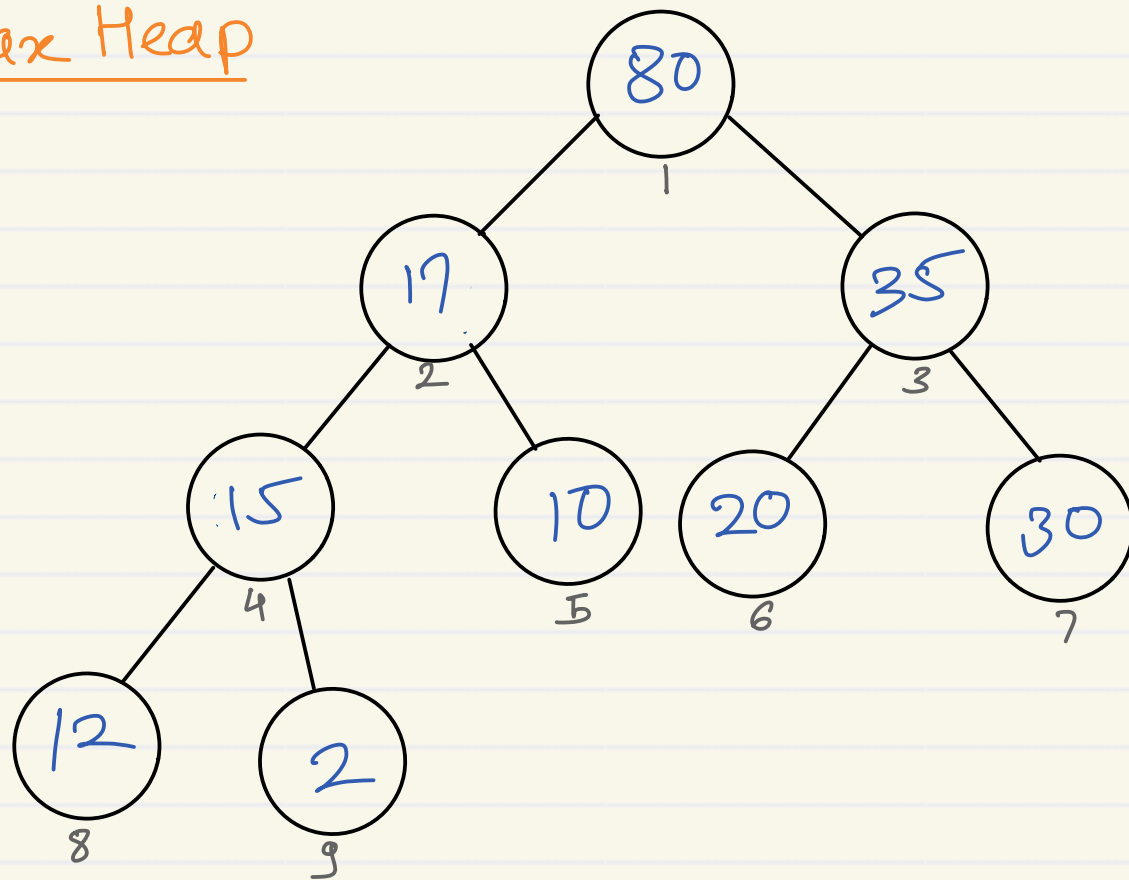


10	30	20	35	40	32	25
1	2	3	4	5	6	7

- Min heap is a heap data structure in which each node is smaller than both of its child nodes.



Max Heap



Keys : 20, 12, 35, 15, 10, 80, 30, 17, 2

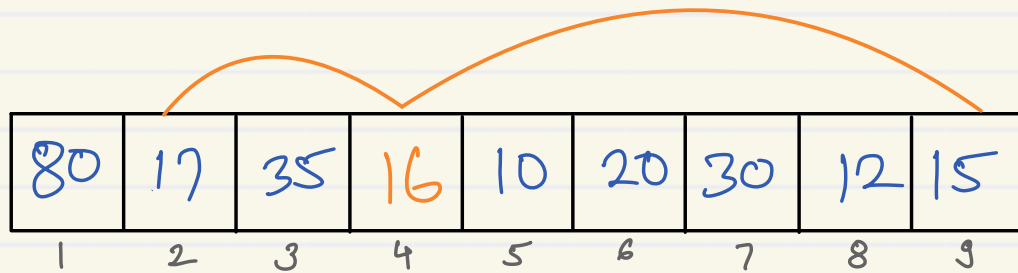
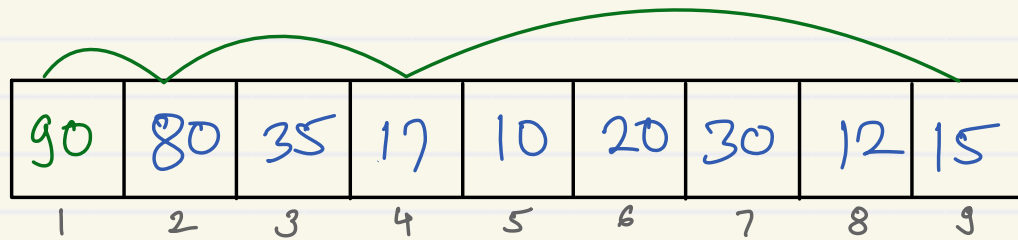
Algorithm:

1. add new value on first empty index from left side
2. adjust position of newly added value (compare with ancestors)

- to add value into heap, need to traverse from leaf to root position

Time $\propto$ height

$$T(n) = O(\log n)$$



value = 90

c_i	p_i
9	4
4	2
2	1
1	0

↑
not valid index

value = 16

c_i	p_i
9	4
4	2

↑
parent is already maximum

c_i = index of newly added value

$p_i = c_i / 2$

while($p_i > 0$) {

if (parent is maximum)

break;

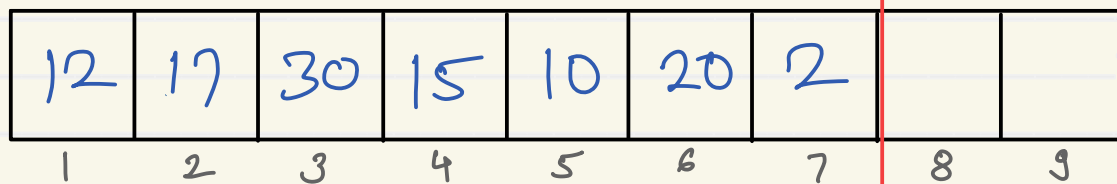
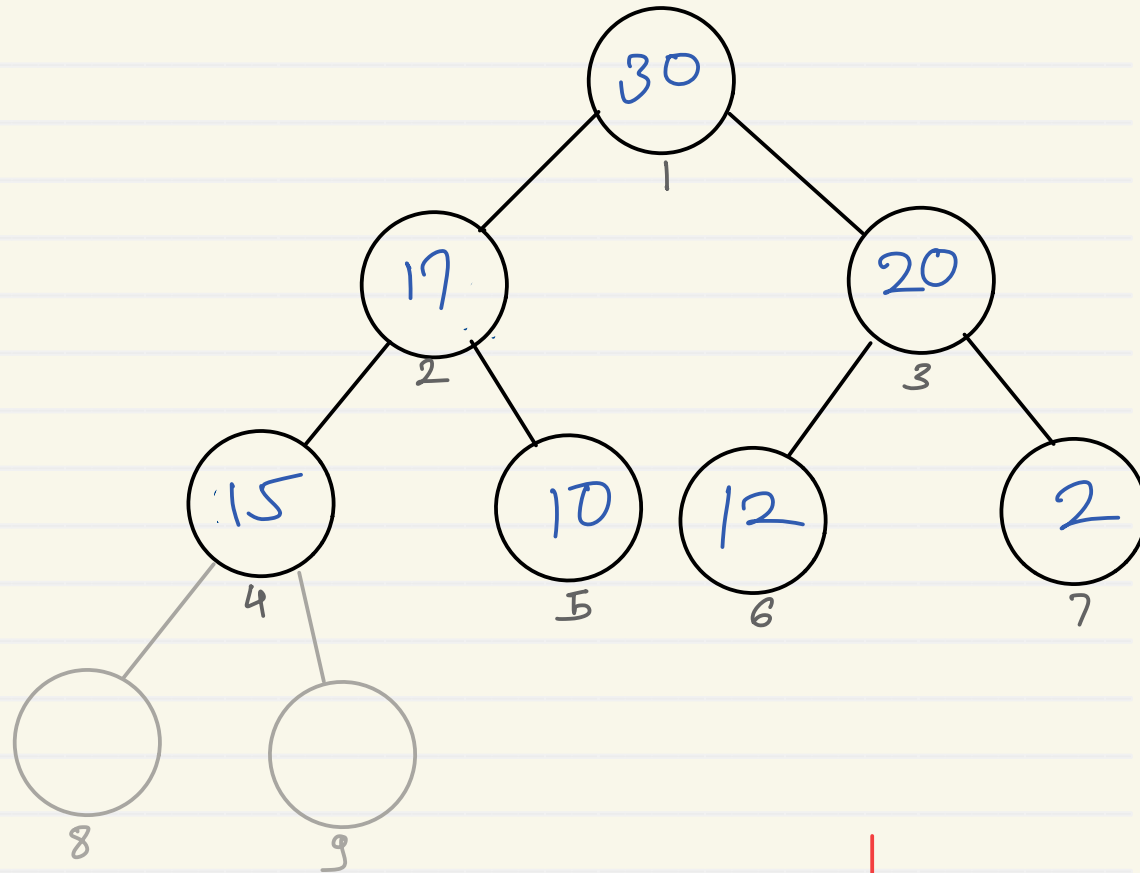
swap

$c_i = p_i$

$p_i = c_i / 2$;

}

Heap - Delete heap



Property: only root element can be deleted

Max heap: always maximum value is removed

Min heap: always minimum value is removed

Max value: 80, 35

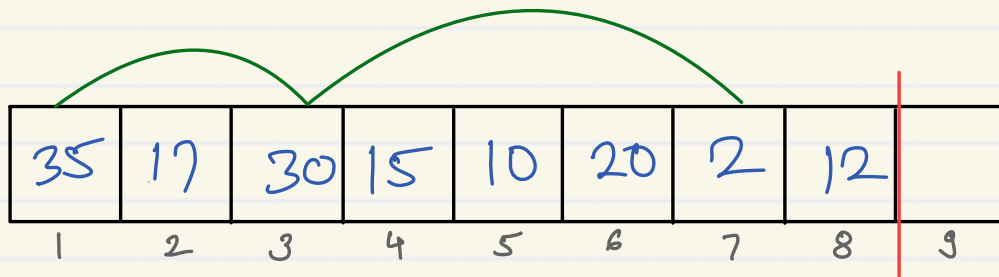
Algorithm:

1. After deleting root, promote last element of heap to the root place
2. Adjust position of promoted element (compare with descendants)

- to delete element from heap need to traverse from root to leaf position

Time $\propto$ height

$$T(n) = O(\log n)$$



Max = 80

P_i C_i

1 2, 3

3 6, 7

7 14

↑ invalid child index

left child = $P_i \times 2$ = C_i

right child = $P_i \times 2 + 1$ = $C_i + 1$

```

P_i = 1;
C_i = P_i * 2;
while (C_i <= size) {
    C_i = find index of max child
    - if (parent is maximum than max child)
        break;
    - swap parent & max child
    P_i = C_i;
    C_i = P_i * 2;
}
    
```

- Queue where high priority data is always peeked or deleted.
- Priority can be implemented using array, linked list and heap data structures.
- An array is used to store a value and its associated priority. In some simpler implementations, the value itself might represent the priority (e.g., lower value means higher priority).
- Array and linked list implementation of priority queue often leads to less efficient performance compared to heap-based implementations for insertion and deletion operations.

- **Ordered vs. Unordered Array**

- **Ordered Array / *linked list***

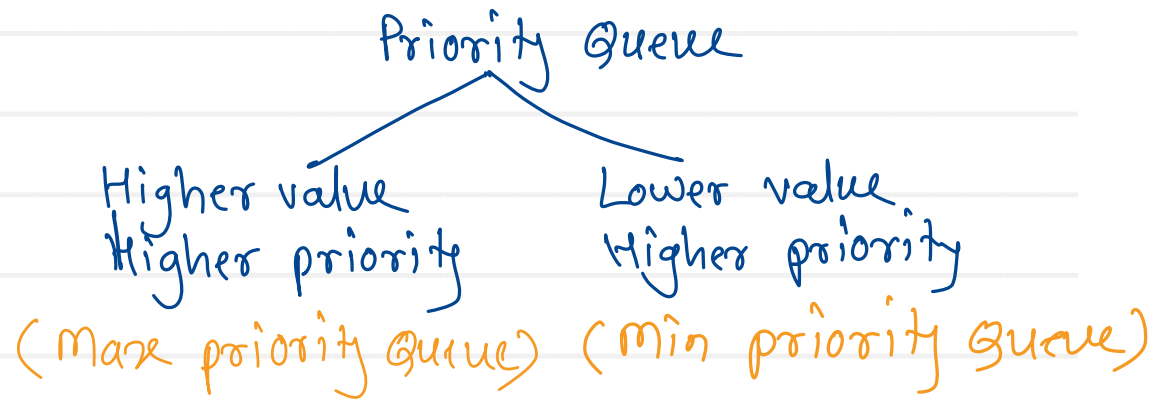
- Elements are kept sorted by priority. Insertion requires shifting existing elements to maintain order, leading to $O(n)$ time complexity for insertion.
 - Deletion of the highest priority element is $O(1)$ as it's typically at the beginning or end of the array.

- **Unordered Array: *linked list***

- Elements are inserted without regard to order, making insertion $O(1)$.
 - Deletion of the highest priority element requires searching the entire array to find it, resulting in $O(n)$ time complexity for deletion.

Priority Queue Implementation

- Priority : number associated with value
- Priority range is defined by programmer
e.g. Priority range : 1 to 10



- Every element of priority queue will have two parts:

value : any data type
priority : integer

```
class item {
    int value;
    int priority;
};
```

```
class priorityQueue {
    class item arr[5];
    int capacity;    (maxSize)
    int size;
};
```

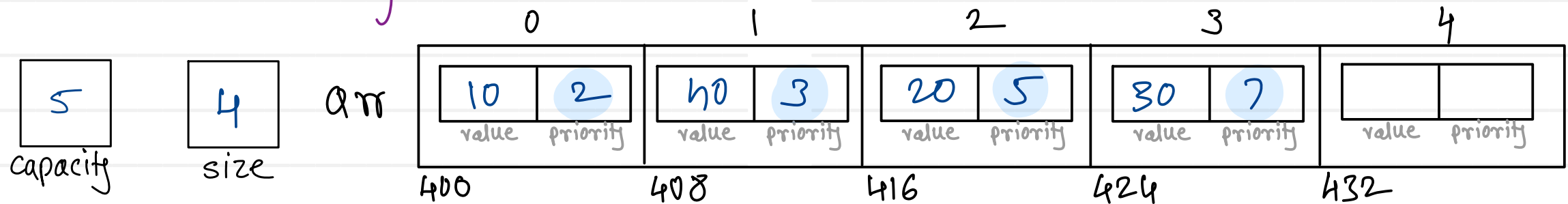
Operations :

- 1) Enqueue
- 2) Dequeue
- 3) Peek

- 4) isEmpty $\rightarrow$ size == 0
- 5) isFull $\rightarrow$ size == capacity

Priority Queue Implementation

Ordered array:



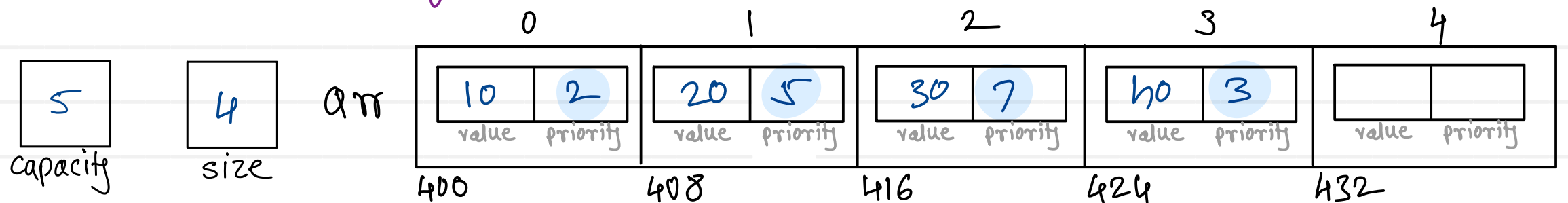
insert - $O(n)$

delete - $O(1)$

peek - $O(1)$

for shifting - $O(n)$

Unordered array:



insert - $O(1)$

delete - $O(n)$

peek - $O(n)$

for shifting - $O(n)$